Software Engineering - Lab 3

Name : Nishkaa V

Topic : Community Lost & Found Matching Network

SRN : PES2UG24CS907

Class : 5 - B

Architecture Selection: I chose Microservices Architecture for the Community Lost & Found Matching Network System.

Two Reasons for Selection:

1.Targeted Scalability for Computational Workloads: The matching engine requires continuous processing to compare new entries, map spatial coordinates, and analyze image metadata. By isolating it as an independent microservice, compute resources can be dynamically scaled during high-traffic events (e.g., localized surges in lost item reports) without degrading the performance or availability of the core application.

2.Enhanced Fault Isolation: Integrations with third-party APIs for external communications (such as SMS or push notifications) introduce latency and reliability risks. Encapsulating the notification service within a dedicated microservice ensures that external gateway failures or timeouts do not cascade. Consequently, core functionalities such as user authentication, item registration, and feed browsing, remain fully operational.

Security Advantage: A microservices architecture facilitates stringent data segregation and adherence to the principle of least privilege. The Matching Engine operates strictly on anonymized item identifiers and geospatial bounding boxes. Concurrently, sensitive Personally Identifiable Information (PII) is isolated entirely within the User & Auth component. This compartmentalization significantly reduces the attack surface, ensuring that user contact details remain secure even if public-facing item feeds are compromised.

Performance Benefit: This architectural style inherently supports asynchronous processing, minimizing thread-blocking operations on the main application. When a user uploads a record with high-resolution media, the intensive matching request is offloaded to backend services. Because these algorithms execute in the background, the primary

server is freed up immediately, maintaining a highly responsive, low-latency user interface.

Component Diagram

